

Exercise 3b Processes in UNIX

Learning objectives:

- Start, stop, and determine processes in the foreground and background on a shell in a Unix system
- Explain terms like parent, child process
- Create or call a process from an application program

Legend:

 Reading to be completed before the exercise

 Questions/tasks to be completed before the exercise

 Tasks to be worked on during the exercise

3.1 Exercise Preparation

The preparations and questions should be completed before the exercise session. The exception is this first exercise, where you haven't had the opportunity to read the material yet. They are also intended as aids to make the tasks to be worked on during the exercise easier for you.

 Study the following materials in preparation:

- Chapter 8.4 + 8.5 of Chr. Baun, Operating Systems, 2nd Edition,
- Video "Linux/Unix Process Management: ps, kill, top, df, free, nice Commands"
- Commands for Process Management in Linux
- fork() and exec() in Introduction of Computer Structures

The following material is in German only:

- Chapter 7 in "Linux-Unix Programming"
- Chapter 4.1.5 in "System Programming in UNIX/Linux. Vieweg+Teubner Verlag"

 **Introductory Questions:**

Which of the following commands in the shell can list **all** currently running and runnable processes of all system users?

- ☐ showproc
- ☐ ps
- ☐ pstree
- ☐ ps -a
- ☐ top
- ☐ ps -ef
- ☐ fg
- ☐ bg
- ☐ nice
- ☐ ps -aux

How do you start any shell command in the background?

Which shell command can be used to stop a process?

Which shell command can be used to *terminate* a process from the system?

What does the system command fork return?

Research and explain what the commands `fork`, `execl`, `wait`, and `exit` do. Also, compare these commands with each other.

Look at the C program in the following subtask ??.

3.2 Exercises during the exercise session

❖ UNIX shell commands

Use the man pages to clarify questions about the parameters of the following UNIX commands, if necessary. At first, you should run two terminals. Start a text editor by typing in the command `mousepad` (on Raspi) or `gedit` (on Ubuntu and other Linux Distros) in the foreground in one of the terminals

- Check how many and which processes are currently running in your environment. What command must be used for that?
- Stop the process of your text editor in the second shell using `kill -s STOP <PID>`. What process state does `mousepad` or `gedit` take now? What happens when you try to use the text editor?

- Set `mousepad` or `gedit` back to a running background process. What state information do you now get about this process? What happens when you enter text into the editor?



3.3 Creating a New Process under UNIX

 As preparation, you should already be familiar with the functionality of the following program.

❖ Compile the following C program.

```
/*
parent_and_child.c
```

Parent and child processes output their PID and the PID of their parent process. The fork function is used to create a child process.

The two processes operate independently of each other, which means the output can be mixed up!

This can be prevented with semaphores, which will be covered later.

The outputs are clearly assigned to a process, the parent or the child, by using a "P" or "C".

We use the two system functions getpid and getppid to query the PID of a process and that of its parent process.

These are defined in <unistd.h>.

```
*/
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
```

```
int main()
{
    int pid_child = 0;
    int counter = 0;
    printf("P is for Parent\n");
    printf("C is for Child\n");
    /* fork returns the process ID of the child process,
     * 0 for this process or < 0 in case of an error.
     * This value is assigned to the respective Variable.
     */
    pid_child = fork();

    if (pid_child < 0)
    { /* error handling leads to program exit */
```

```
fprintf(stderr, "\n fork did not work!\n ");
exit(EXIT_FAILURE);

}
/* Child process */
if (pid_child == 0)
{
    /* Doing some counter increment .... */
    while (counter < 10) /* change the max value here .... */
    {
        counter = counter + 1;
        {
            printf("C: counter = %d\n", counter);
        }
    }
    printf("C: ID of this process is: %i\n", getpid());
    printf("C: ID of the parent process is: %i\n", getppid());
    printf("C: Child process ends. \n");
}
/* Parent process */
if (pid_child > 0)
{
    /* Doing some counter increment .... */
    while (counter < 10) /* change the max value here .... */
    {
        counter = counter + 1;
        {
            printf("P: counter = %d\n", counter);
        }
    }

    // wait(0);
    printf("P: ID of this process is: %i \n", getpid());
    printf("P: ID of the parent process is: %i \n", getppid());

    printf("P: The parent process ends. \n");
}

return(EXIT_SUCCESS);
}
```

- a) What do you notice regarding the PIDs of the parent and child processes when executing the program? Which parent process is now indicated for the child process?

- b) Now change the maximum number of loop iterations for both the parent process. Later on also for the child process. What do you notice while running the programs with various maximum values? Which parent process is now indicated for the child process? What difficulty could arise if parent and child processes were to exchange data?

3.4 Process Synchronization

❖ In the task 3.3, you may have already discovered the particular situation for orphaned child processes.

- a) Which C system command could you use to solve the issue? Research the command or commands that could prevent getting orphaned child processes.

- b) Modify the given program from listing 3.3 to fix the problem.

3.5 Start and Wait for an Arbitrary Process

❖ In this exercise, you have to write a C program that lists all files and directories in the `/home/student` directory. If you use another Linux-System, use the Home-Directory of your standard user.

- a) You may have already researched the parameters for the C system function call `execl`.
- b) Write the desired C program using `eclipse`.